

C Functions

A function in C is a set of statements that when called perform some specific tasks. The programming statements of a function are enclosed within { } braces, having certain meanings and performing certain operations.

Every C program includes at least one function, typically the main () function, which serves as the entry point for execution. Functions can take parameters and return values, allowing them to be versatile and reusable across different parts of a program. They are also called subroutines or procedures in other languages.

Importance of Functions in C

1. Modularity

Functions promote modular programming by dividing a program into distinct sections. Each function can handle a specific task, making the overall program structure clearer and more organized. This modularity simplifies both development and debugging processes.

2. Reusability

Once defined, functions can be reused multiple times throughout a program without needing to rewrite code. This not only saves time but also reduces redundancy, leading to more efficient coding practices.

3. Readability

By breaking complex code into smaller functions, programs become easier to read and understand. This is particularly beneficial for larger projects where maintaining clarity is crucial for collaboration among multiple developers.

4. Easier Debugging

Functions help isolate specific tasks within a program, making it easier to identify and fix errors. When issues arise, developers can focus on individual functions rather than sifting through extensive code.

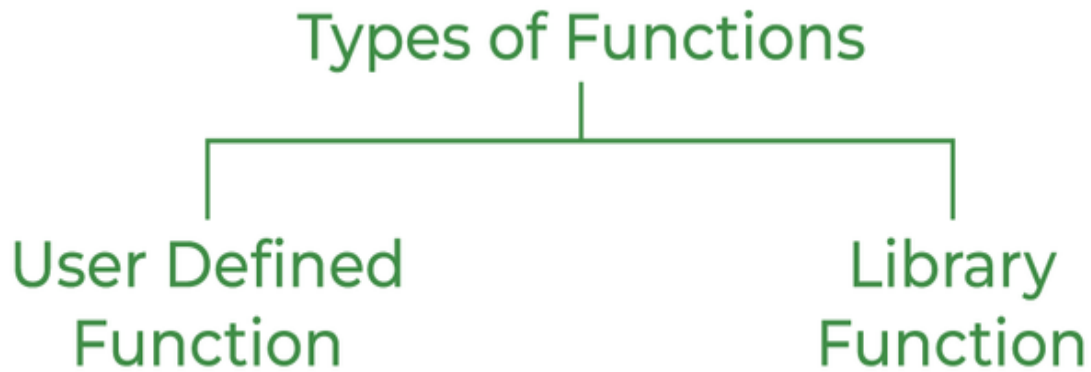
5. Abstraction

Functions provide a level of abstraction by encapsulating specific functionalities. This means that users of the function do not need to understand its internal workings; they only need to know how to call it and what it returns.

6. Efficiency

Functions can be optimized individually, which can lead to overall performance improvements in the program. By focusing on optimizing specific functions rather than the entire program at once, developers can achieve better efficiency.

Types of Functions



1. Library Function

A library function is also referred to as a “built-in function”. A compiler package already exists that contains these functions, each of which has a specific meaning and is included in the package. Built-in functions have the advantage of being directly usable without being defined, whereas user-defined functions must be declared and defined before being used.

For Example:

`pow()`, `sqrt()`, `strcmp()`, `strcpy()` etc.

Example:

// C program to implement the above approach

```
#include <math.h>
```

```
#include <stdio.h>
```

```
// Driver code
```

```
int main(){
```

```
double Number;
```

```
Number = 49;
```

```
// Computing the square root with the help of predefined C library function
```

```
double squareRoot = sqrt(Number);
```

```
printf("The Square root of %.2lf = %.2lf", Number, squareRoot);
```

```
return 0;}
```

Output

The Square root of 49.00 = 7.00

2. User Defined Function

Functions that the programmer creates are known as User-Defined functions or “tailor-made functions”. User-defined functions can be improved and modified according to the need of the programmer. User-defined functions can be further categorized based on their parameters and return values:

1. **Functions with Arguments and Return Values:** These functions take parameters and return a value. They are often used for calculations or transformations.
2. **Functions with Arguments and Without Return Values:** These functions accept parameters but do not return any value, often used for operations that do not require a result to be sent back.
3. **Functions without Arguments and With Return Values:** These functions do not take any parameters but return a value, useful for simple computations that do not require input.
4. **Functions without Arguments and Without Return Values:** These functions neither take parameters nor return values, typically used for performing an action without needing input or output.

```
// C program to show user-defined functions
#include <stdio.h>
int sum(int a, int b) {
    return a + b; }
```

```
// Driver code
int main(){
    int a = 30, b = 40;
    // function call
    int res = sum(a, b);
    printf("Sum is: %d", res);
    return 0;}
```

Output

Sum is: 70

Function Declarations

A function declaration serves as a promise to the compiler that a function exists with a specified name, return type, and parameters. It does not include the actual implementation of the function. The primary purpose of declaring a function is to inform the compiler about its signature so that it can be called before its definition appears in the code.

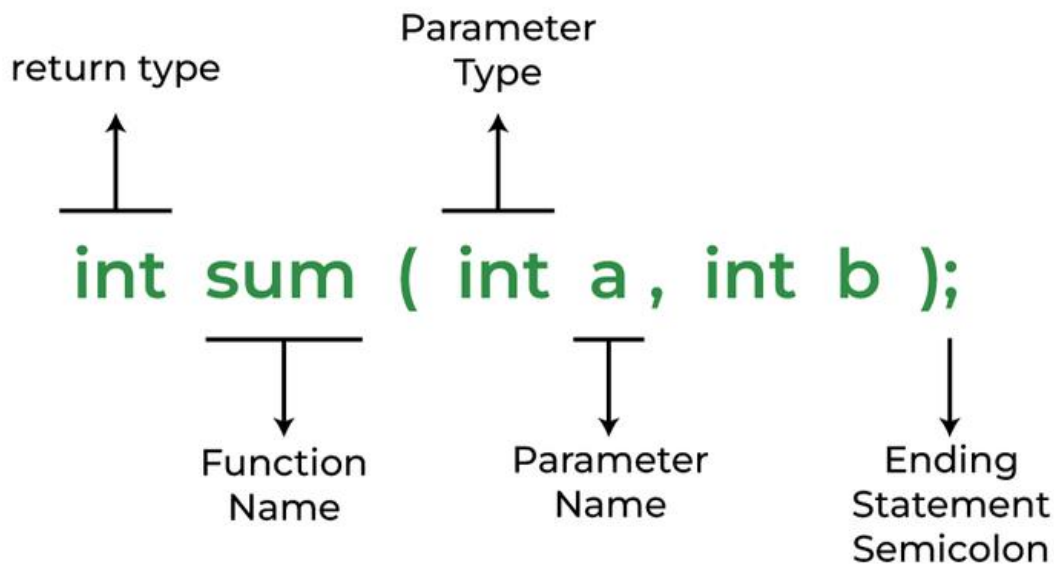
Syntax

```
return_type name_of_the_function (parameter_1, parameter_2);
```

The parameter name is not mandatory while declaring functions. We can also declare the function without using the name of the data variables.

Example

```
int sum(int a, int b); // Function declaration with parameter names  
int sum(int , int);    // Function declaration without parameter names
```



Note: A function in C must always be declared globally before calling it.

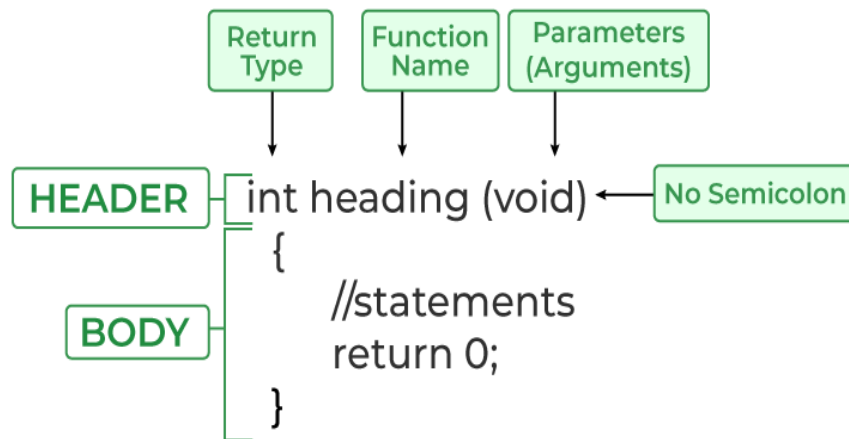
Function Definition

A function definition, on the other hand, provides the actual implementation of the function. This includes the body of the function, which contains the code that will be executed when the function is called. The definition must match the declaration in terms of name, return type, and parameters.

Syntax:

```
return_type function_name (para1_type para1_name, para2_type para2_name)
{
    // body of the function
}
```

Function Definition



```
// C program to show function call, declaration and definition
#include <stdio.h>
```

```
//Function Declaration with return type, name and parameters
int sum(int , int );
```

```
// Driver code
int main(){
    // Calling sum function and storing its value in add variable
    int add = sum(10, 30);
    printf("Sum is: %d", add);
    return 0;}
```

```
// Function that takes two parameters a and b as inputs and returns their sum
int sum(int a, int b)
{
    return a + b; }
```

Output

Sum is: 40

Function Arguments

Function arguments (also called parameters) are the values or variables passed into a function when it's called. These arguments allow a function to receive input, which can be used within the function's body to perform calculations or operations. When you declare a function, you specify the number and data type of arguments it expects within parentheses ().

Example:

```
int add(int a, int b);
```

 (This function add takes two integer arguments a and b.)

When you call a function, you provide the actual values for the arguments.

Example:

```
int sum = add(5, 3);
```

 (Here, 5 is passed to a, and 3 is passed to b.)

Types of Arguments

Actual Arguments: The values that are passed to the function during the function call. (e.g., 5 and 3 in the example above)

Formal Arguments: The parameters that are defined in the function declaration/definition. (e.g., int a and int b in the example above)

Return Type

A function can return a value back to the caller after it has finished executing. The return type is specified before the function name in the function declaration.

Example:

```
int add(int a, int b);
```

 (This declares that the function add will return an integer value.)

return Statement: Inside the function definition, the return statement is used to send a value back.

Example:

```
int add(int a, int b) {  
    int sum = a + b;  
    return sum; // Returns the calculated sum  
}
```

void Return Type: If a function does not need to return any value, you use the void keyword as the return type.

Example:

```
void printMessage(char message[]);
```

 (This function does not return anything.)

Example with Arguments and Return Type

```
#include <stdio.h>
```

```
// Function declaration
```

```
int multiply(int x, int y);
```

```
int main() {
```

```
    int num1 = 10;
```

```
    int num2 = 20;
```

```
    int result = multiply(num1, num2); // Function call with arguments
```

```
    printf("The product of %d and %d is %d\n", num1, num2, result);
```

```
    return 0;
```

```
}
```

```
// Function definition
```

```
int multiply(int x, int y) {
```

```
    return x * y; // Returns the product of x and y
```

```
}
```

Key Points

- A function can have zero or more arguments.
- A function can have only one return value. If you need to return multiple values, you can use pointers, arrays, or structures.
- If no return type is specified, C compilers assume it to be int (though it's best practice to always specify the return type explicitly).

How to Call a Function

To call a function in C, you use the function's name followed by parentheses (). If the function takes arguments, you provide the values for those arguments inside the parentheses.

```
#include <stdio.h>
```

```
// Function declaration
```

```
int add(int a, int b);
```

```
int main() {
```

```

int num1 = 5;
int num2 = 3;
int sum = add(num1, num2); // Function call

printf("The sum is: %d\n", sum);
return 0;
}

// Function definition
int add(int a, int b) {
    return a + b;
}

```

In this example, `add(num1, num2)` is the function call.

Types of Function Calls in C

There are two main ways to call a function in C, distinguished by how arguments are passed:

1. Call by Value

- When you call a function "by value," a copy of the actual argument's value is passed to the function's formal parameter.
- Any changes made to the formal parameter inside the function do not affect the original variable in the calling function.

Example:

```

#include <stdio.h>

void modify(int x) {
    x = x * 2;
    printf("Value inside function: %d\n", x);
}

int main() {
    int num = 10;
    modify(num); // Call by value
    printf("Value in main: %d\n", num);
    return 0;
}

```

Output:

```
// Value inside function: 20
```

```
// Value in main: 10
```

As you can see, even though `x` was modified inside the `modify` function, the value of `num` in `main` remained unchanged.

2. Call by Reference

- When you call a function "by reference," the memory address of the actual argument is passed to the function's formal parameter.
- The formal parameter acts as an alias for the actual argument. Therefore, any changes made to the formal parameter inside the function *do* affect the original variable in the calling function.
- In C, call by reference is achieved using pointers.

Example:

```
#include <stdio.h>
```

```
void modify(int *x) { // x is a pointer
    *x = (*x) * 2; // Dereference x to access the value at that address
    printf("Value inside function: %d\n", *x);
}
```

```
int main() {
    int num = 10;
    modify(&num); // Call by reference (pass the address of num)
    printf("Value in main: %d\n", num);
    return 0;
}
```

Output:

```
// Value inside function: 20
```

```
// Value in main: 20
```

Here, because we passed the address of `num` to the `modify` function, modifying `*x` inside the function directly changed the value of `num` in `main`.

Choosing Between Call by Value and Call by Reference

- Use **call by value** when you don't need to modify the original variable in the calling function. This is generally safer as it prevents unintended side effects.
- Use **call by reference** when you need to modify the original variable or when you need to pass large data structures (like arrays or structs) to avoid the overhead of copying.

Recursion in C

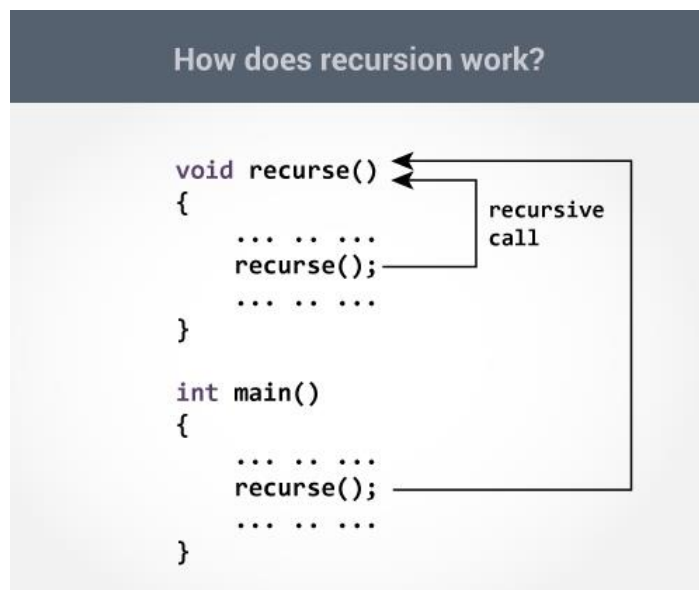
Recursion in C is a programming technique where a function calls itself either directly or indirectly to solve a problem. Recursive functions are commonly used to solve problems that can be broken down into smaller, similar sub-problems. Each recursive call should bring the problem closer to a base case, which is the condition that terminates the recursion.

Key Components of a Recursive Function

1. **Base Case:** The condition that stops the recursion to prevent infinite recursion.
2. **Recursive Case:** The part of the function where the function calls itself with modified arguments.

General Syntax of a Recursive Function

```
return_type function_name(parameters) {  
    if (base_case_condition) {  
        // Base case logic  
        return some_value;  
    } else {  
        // Recursive case logic  
        return function_name(modified_parameters);  
    }  
}
```



Example: Sum of Natural Numbers Using Recursion

```
#include <stdio.h>

int sum(int n);

int main() {
    int number, result;
    printf("Enter a positive integer: ");
    scanf("%d", &number);

    result = sum(number);
    printf("sum = %d", result);
    return 0;}

int sum(int n) {
    if (n != 0)
        return n + sum(n-1);
    else
        return n;}
```

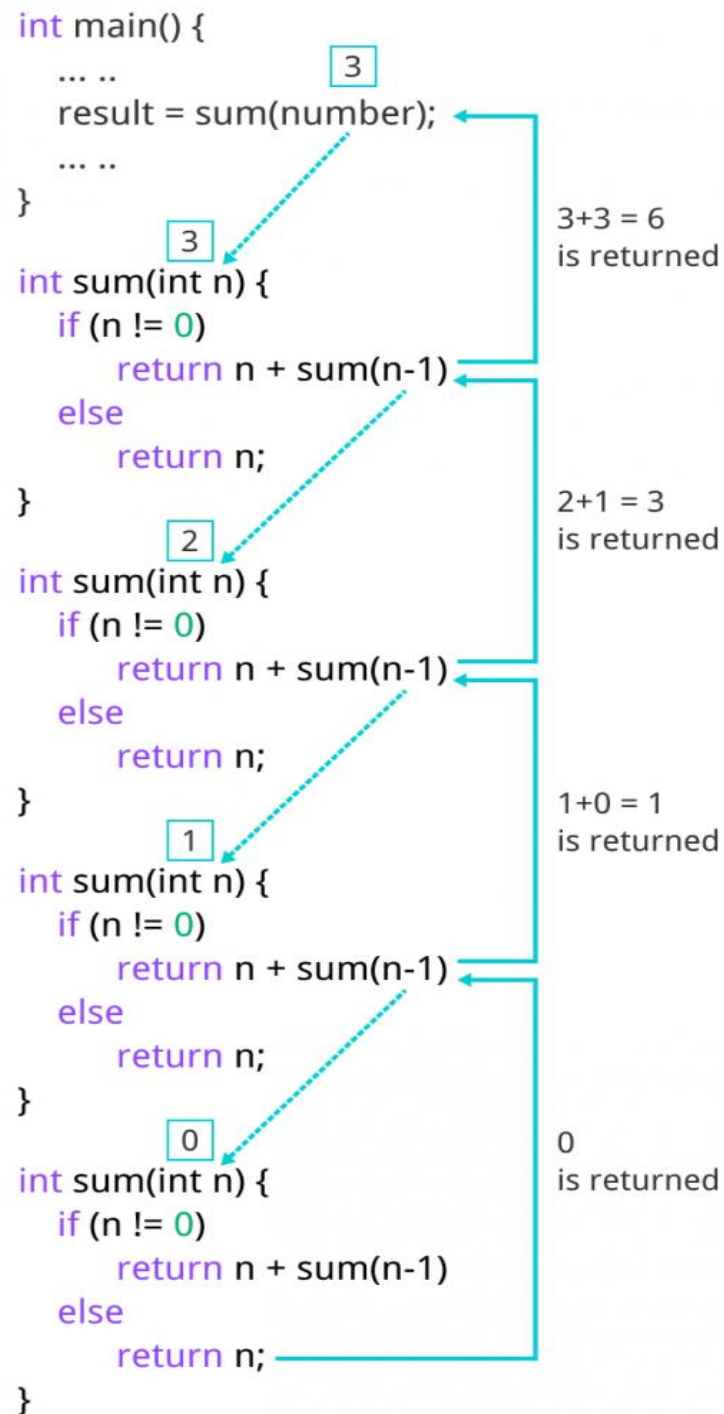
Output

```
Enter a positive integer:3
sum = 6
```

Initially, the sum() is called from the main() function with number passed as an argument.

Suppose, the value of n inside sum() is 3 initially. During the next function call, 2 is passed to the sum() function. This process continues until n is equal to 0.

When n is equal to 0, the if condition fails and the else part is executed returning the sum of integers ultimately to the main() function.



Example: Factorial of a Number using recursion

```
#include <stdio.h>
```

```
int factorial(int n) {  
    if (n == 0) // Base case  
        return 1;  
    else  
        return n * factorial(n - 1); // Recursive call  
}  
  
int main() {  
    int num = 5;  
    printf("Factorial of %d is %d\n", num, factorial(num));  
    return 0;  
}
```

How it Works: Factorial is the product of all positive numbers from 1 to the given number (n). Example: Factorial of 5 = $5 \times 4 \times 3 \times 2 \times 1 = 120$

- The **base case** is $n == 0$, where the factorial of 0 is defined as 1. This stops the recursion.
- For other values of n, the function calls itself with a smaller number (n-1), multiplying n with the result of the smaller problem.

Step-by-Step Execution for factorial (5):

1. factorial(5) calls $5 * \text{factorial}(4)$
2. factorial(4) calls $4 * \text{factorial}(3)$
3. factorial(3) calls $3 * \text{factorial}(2)$
4. factorial(2) calls $2 * \text{factorial}(1)$
5. factorial(1) calls $1 * \text{factorial}(0)$
6. factorial(0) returns 1 (base case)
7. Now the results are multiplied step-by-step:
 - $1 * 1 = 1$
 - $2 * 1 = 2$
 - $3 * 2 = 6$
 - $4 * 6 = 24$
 - $5 * 24 = 120$

Output:

Factorial of 5 is 120

Example: Fibonacci Series using recursion

```
#include <stdio.h>
```

```
int fibonacci(int n) {  
    if (n == 0) // Base case  
        return 0;  
    else if (n == 1) // Base case  
        return 1;  
    else  
        return fibonacci(n - 1) + fibonacci(n - 2); // Recursive call  
}  
  
int main() {  
    int terms = 5;  
    printf("Fibonacci Series: ");  
  
    for (int i = 0; i < terms; i++) {  
        printf("%d ", fibonacci(i));  
    }  
  
    return 0;}
```

How it Works: The **Fibonacci series** is a sequence of numbers where each number is the sum of the two preceding numbers. Example: 0, 1, 1, 2, 3, 5, 8, 13 ...

1. The **base cases** are:

- fibonacci(0) returns 0.
- fibonacci(1) returns 1.

2. For larger numbers, the function calculates the sum of fibonacci(n-1) and fibonacci(n-2).

Step-by-Step Execution for fibonacci(5):

1. fibonacci(5) calls fibonacci(4) + fibonacci(3)
2. fibonacci(4) calls fibonacci(3) + fibonacci(2)
3. fibonacci(3) calls fibonacci(2) + fibonacci(1)
4. This process continues until it reaches the base cases (fibonacci(0) and fibonacci(1)).
5. The results are then added together step-by-step to get the Fibonacci number for n=5.

Output:

Fibonacci Series:

0 1 1 2 3 5 8 13 21 34

Advantages of Recursion

1. Simplifies code for problems that are inherently recursive (e.g., factorial, Fibonacci, tree traversals).
2. Reduces the need for complex loops or stack management.

Disadvantages of Recursion

1. Can lead to stack overflow if the recursion depth is too large.
2. Generally less efficient than iterative solutions due to overhead in function calls.
3. Requires careful handling of the base case to avoid infinite recursion.